Day 4 handout

Day 4: Numerical Computing Foundations

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

Overview

This handout covers the fundamental libraries for numerical computing in Python: `numpy`, `scipy`, and `matplotlib`. These form the **core stack** for scientific data analysis and are essential tools for modern research computing.

Learning objectives:

- Understand NumPy arrays and their advantages over Python lists
- Master array creation, indexing, slicing, and manipulation
- Apply NumPy's mathematical and statistical functions
- Use SciPy for specialized scientific computing tasks
- Create effective data visualizations with matplotlib
- Integrate these tools for complete data analysis workflows

Prerequisites: Basic Python knowledge (variables, functions, control flow, data types)

Part 1: NumPy - Numerical Python

1.1 What is NumPy?

NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides:

- **N-dimensional array object (ndarray)**: Efficient storage and operations
- **Broadcasting**: Implicit element-wise operations on arrays of different shapes
- **Mathematical functions**: Comprehensive library of mathematical operations
- **Linear algebra tools**: Matrix operations, decompositions, solving linear systems
- **Random number capabilities**: Various distributions and sampling methods
- **Integration with compiled code**: Interface to C/C++/Fortran libraries

> NumPy forms the foundation for most scientific Python packages, including SciPy, pandas, scikit-learn, and many domain-specific libraries.

Standard import convention:

```
import numpy as np
print(np.__version__)  # Check version (e.g., '1.26.0' or newer)
```

1.2 Why NumPy? Performance and Memory Efficiency

Python lists vs NumPy arrays:

Feature	Python Lists	NumPy Arrays
Data types	Heterogeneous (mixed types)	Homogeneous (single type)
Memory	Each element is a full Python object	Contiguous memory blocks
Speed	Slow for numerical operations	50-100x faster
Size	High overhead per element	Minimal overhead
Operations	Element-by-element (loops)	Vectorized operations

Performance example:

```
import numpy as np
import time

# Python list approach
n = 1000000
python_list = list(range(n))

start = time.time()
result_list = [x**2 for x in python_list]
list_time = time.time() - start

# NumPy array approach
numpy_array = np.arange(n)

start = time.time()
result_array = numpy_array**2
numpy_time = time.time() - start

print(f"List time: {list_time:.4f}s")
print(f"NumPy time: {numpy_time:.4f}s")
print(f"Speedup: {list_time/numpy_time:.1f}x")
# Typical output: ~50-100x faster with NumPy
```

Memory comparison:

```
import sys

# Python list
python_list = list(range(1000))
list_memory = sys.getsizeof(python_list) + sum(sys.getsizeof(x) for x
in python_list)
```

```
# NumPy array
numpy_array = np.arange(1000)
array_memory = numpy_array.nbytes

print(f"List memory: {list_memory:,} bytes")
print(f"Array memory: {array_memory:,} bytes")
print(f"Memory reduction: {list_memory/array_memory:.1f}x")
# NumPy typically uses 5-10x less memory
```

1.3 NumPy vs MATLAB

For researchers transitioning from MATLAB, here are key differences:

Feature	MATLAB	NumPy/Python
Indexing	Starts at 1: `A(1)`	Starts at 0: `A[0]`
Last element	`A(end)`	`A[-1]`
Slicing	`A(1:5)` includes 5	`A[0:5]` excludes 5
Element-wise mult	`A .* B`	`A * B`
Matrix mult	`A * B`	`A @ B` or `np.dot(A, B)`
Power	`A.^2`	`A**2`
Default type	double (float64)	Depends on data
Size function	`size(A)`	`A.shape`
Cost	Commercial license	Free and open source

Common MATLAB to NumPy translations:

```
# MATLAB: A = [1 2 3; 4 5 6]
A = np.array([[1, 2, 3], [4, 5, 6]])

# MATLAB: B = zeros(3, 4)
B = np.zeros((3, 4))

# MATLAB: C = ones(2, 5)
C = np.ones((2, 5))

# MATLAB: D = rand(3, 3)
D = np.random.rand(3, 3)

# MATLAB: E = linspace(0, 10, 100)
E = np.linspace(0, 10, 100)

# MATLAB: F = A'
F = A.T

# MATLAB: G = inv(S)  (only square matrices can be inverted)
S = np.array([[2, 1], [1, 3]])
G = np.linalg.inv(S)
```

```
# MATLAB: H = A * B (matrix multiplication)
H = A @ B  # or np.dot(A, B)
```

Reference: https://numpy.org/doc/stable/user/numpy-for-matlab-users.html

1.4 Creating NumPy Arrays

1.4.1 From Python Lists and Tuples

```
# 1D array from list
a = np.array([1, 2, 3, 4, 5])
print(a)  # [1 2 3 4 5]
print(type(a))  # <class 'numpy.ndarray'>

# 2D array from nested lists
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b)
# [[1 2 3]
#  [4 5 6]]

# 3D array
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
print(c.shape)  # (2, 2, 2)
```

1.4.2 Using Built-in Functions

```
# Array of zeros
zeros = np.zeros(5)              # [0. 0. 0. 0. 0.]
zeros_2d = np.zeros((3, 4))      # 3x4 array of zeros

# Array of ones
ones = np.ones(5)                # [1. 1. 1. 1. 1.]
ones_2d = np.ones((2, 3))        # 2x3 array of ones

# Array of specific value
fives = np.full(5, 5.0)          # [5. 5. 5. 5. 5.]
fives_2d = np.full((2, 3), 5.0)  # 2x3 array of 5.0

# Identity matrix
identity = np.eye(3)             # 3x3 identity matrix

# Array from range
range_array = np.arange(0, 10, 2)  # [0 2 4 6 8]
range_float = np.arange(0.0, 1.0, 0.1)  # [0.  0.1 0.2 ... 0.9]

# Linearly spaced values
linear = np.linspace(0, 10, 5)   # [0.  2.5  5.  7.5 10.]
```

```
# Logarithmically spaced values
log_space = np.logspace(0, 3, 4) # [1.e+00 1.e+01 1.e+02 1.e+03]

# Empty array (uninitialized - fastest)
empty = np.empty(5)              # Random values (whatever was in
memory)
```

1.4.3 Random Arrays

```
# Set seed for reproducibility
np.random.seed(42)

# Uniform random values [0, 1)
uniform = np.random.rand(3, 3)

# Uniform random integers
integers = np.random.randint(0, 10, size=(3, 3))

# Normal distribution (mean=0, std=1)
normal = np.random.randn(5)

# Normal distribution (custom mean and std)
custom_normal = np.random.normal(loc=100, scale=15, size=10)

# Random choice from array
choices = np.random.choice([1, 2, 3, 4, 5], size=10, replace=True)

# Random permutation
shuffled = np.random.permutation(10)
```

1.5 Array Attributes and Data Types

1.5.1 Key Attributes

```
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])

print(a.ndim)      # 2 (number of dimensions)
print(a.shape)     # (3, 4) (rows, columns)
print(a.size)      # 12 (total number of elements)
print(a.dtype)     # dtype('int64') (data type)
print(a.itemsize)  # 8 (bytes per element)
print(a.nbytes)    # 96 (total bytes = size * itemsize)
```

1.5.2 Data Types (dtype)

```
# Integer types
int8_array = np.array([1, 2, 3], dtype=np.int8)      # -128 to 127
int16_array = np.array([1, 2, 3], dtype=np.int16)    # -32768 to 32767
```

```
        int32_array = np.array([1, 2, 3], dtype=np.int32)    # ~±2 billion
        int64_array = np.array([1, 2, 3], dtype=np.int64)    # ~±9 quintillion

        # Unsigned integer types
        uint8_array = np.array([1, 2, 3], dtype=np.uint8)    # 0 to 255
        uint16_array = np.array([1, 2, 3], dtype=np.uint16)  # 0 to 65535

        # Float types
        float32_array = np.array([1.0, 2.0], dtype=np.float32)  # Single
precision
        float64_array = np.array([1.0, 2.0], dtype=np.float64)  # Double
precision

        # Boolean
        bool_array = np.array([True, False, True], dtype=np.bool_)

        # Complex numbers
        complex_array = np.array([1+2j, 3+4j], dtype=np.complex128)

        # String
        string_array = np.array(['a', 'b', 'c'], dtype='U1')  # Unicode
strings

        # Type conversion
        float_array = np.array([1, 2, 3], dtype=float)
        int_from_float = float_array.astype(int)
```

Choosing the right dtype:

- Use smaller dtypes (int8, float32) to save memory with large arrays
- Use larger dtypes (int64, float64) when precision matters
- Consider memory vs precision tradeoffs in data analysis

1.6 Array Shapes and Reshaping

1.6.1 Reshaping

```
        # Create 1D array
        a = np.arange(12)  # [ 0  1  2  3  4  5  6  7  8  9 10 11]

        # Reshape to 2D
        b = a.reshape(3, 4)
        # [[ 0  1  2  3]
        #  [ 4  5  6  7]
        #  [ 8  9 10 11]]

        # Reshape to 3D
        c = a.reshape(2, 3, 2)

        # Use -1 to auto-calculate dimension
        d = a.reshape(3, -1)   # (3, 4) - automatically calculates 4
```

```
e = a.reshape(-1, 2)   # (6, 2) - automatically calculates 6

# Flatten to 1D (creates copy)
f = b.flatten()

# Ravel to 1D (returns view when possible - faster)
g = b.ravel()
```

Important difference between flatten() and ravel():

```
a = np.array([[1, 2], [3, 4]])
flat = a.flatten()
rav = a.ravel()

# Modify flattened array
flat[0] = 99
print(a)  # [[1 2] [3 4]] - original unchanged (copy)

# Modify raveled array
rav[0] = 99
print(a)  # [[99 2] [3 4]] - original changed (view)
```

1.6.2 Transposing

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)  # (2, 3)

# Transpose
b = a.T
print(b.shape)  # (3, 2)
print(b)
# [[1 4]
#  [2 5]
#  [3 6]]

# Transpose is a view
b[0, 0] = 99
print(a[0, 0])  # 99 - original changed
```

1.6.3 Adding Dimensions

```
a = np.array([1, 2, 3])  # Shape: (3,)

# Add dimension with newaxis
b = a[np.newaxis, :]  # Shape: (1, 3) - row vector
c = a[:, np.newaxis]  # Shape: (3, 1) - column vector

# Or use reshape
d = a.reshape(1, -1)  # Shape: (1, 3)
e = a.reshape(-1, 1)  # Shape: (3, 1)
```

```
# Expand dimensions
f = np.expand_dims(a, axis=0)  # Shape: (1, 3)
g = np.expand_dims(a, axis=1)  # Shape: (3, 1)
```

1.6.4 Stacking and Concatenating

```
a = np.array([[1, 2],
              [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

# Vertical stacking (along rows)
v = np.vstack((a, b))  # or np.concatenate((a, b), axis=0)
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

# Horizontal stacking (along columns)
h = np.hstack((a, b))  # or np.concatenate((a, b), axis=1)
# [[1 2 5 6]
#  [3 4 7 8]]

# Depth stacking (along 3rd dimension)
d = np.dstack((a, b))  # Shape: (2, 2, 2)

# Concatenate along specific axis
c1 = np.concatenate((a, b), axis=0)  # Vertical
c2 = np.concatenate((a, b), axis=1)  # Horizontal

# Stack 1D arrays
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
column_stack = np.column_stack((x, y))  # [[1 4] [2 5] [3 6]]
row_stack = np.vstack((x, y))           # [[1 2 3] [4 5 6]]
(np.row_stack was removed in NumPy 2.x)
```

1.7 Array Indexing and Slicing

1.7.1 Basic Indexing (1D)

```
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Single elements
print(x[0])      # 0 (first element)
print(x[5])      # 5
print(x[-1])     # 9 (last element)
print(x[-2])     # 8 (second to last)
```

```
# Slicing [start:stop:step]
print(x[2:5])    # [2 3 4] (elements 2, 3, 4)
print(x[:5])     # [0 1 2 3 4] (first 5 elements)
print(x[5:])     # [5 6 7 8 9] (from 5 to end)
print(x[::2])    # [0 2 4 6 8] (every 2nd element)
print(x[1::2])   # [1 3 5 7 9] (every 2nd, starting at 1)
print(x[::-1])   # [9 8 7 6 5 4 3 2 1 0] (reversed)
print(x[8:2:-1]) # [8 7 6 5 4 3] (reverse from 8 to 3)
```

1.7.2 Basic Indexing (2D)

```
y = np.array([[0,  1,  2,  3],
              [4,  5,  6,  7],
              [8,  9, 10, 11]])

# Single elements [row, column]
print(y[0, 0])   # 0
print(y[1, 2])   # 6
print(y[-1, -1]) # 11 (last row, last column)

# Entire rows
print(y[0])      # [0 1 2 3] (first row)
print(y[1])      # [4 5 6 7] (second row)
print(y[-1])     # [8 9 10 11] (last row)

# Entire columns
print(y[:, 0])   # [0 4 8] (first column)
print(y[:, 2])   # [2 6 10] (third column)
print(y[:, -1])  # [3 7 11] (last column)

# Subarray slicing
print(y[0:2, 1:3])
# [[1 2]
#  [5 6]]

print(y[:2, :3])
# [[0 1 2]
#  [4 5 6]]

print(y[1:, 2:])
# [[ 6  7]
#  [10 11]]

# Multiple rows/columns
print(y[[0, 2]])  # Rows 0 and 2
# [[ 0  1  2  3]
#  [ 8  9 10 11]]

print(y[:, [0, 3]])  # Columns 0 and 3
# [[ 0  3]
```

```
#  [ 4  7]
#  [ 8 11]]
```

Important: Slices are views, not copies. Modifying a slice modifies the original array:

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]  # View of elements 1, 2, 3
b[0] = 99
print(a)  # [1 99 3 4 5] - original changed!

# To avoid this, make a copy
c = a[1:4].copy()
c[0] = 77
print(a)  # [1 99 3 4 5] - original unchanged
```

1.7.3 Boolean Indexing (Masking)

```
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Create boolean mask
mask = x > 5
print(mask)
# [False False False False False False True True True True]

# Use mask to select elements
print(x[mask])  # [6 7 8 9]

# Direct boolean indexing
print(x[x > 5])      # [6 7 8 9]
print(x[x % 2 == 0]) # [0 2 4 6 8] (even numbers)

# Complex conditions (use & | ~ for and, or, not)
print(x[(x > 2) & (x < 7)])  # [3 4 5 6]
print(x[(x < 3) | (x > 7)])  # [0 1 2 8 9]
print(x[~(x > 5)])           # [0 1 2 3 4 5] (NOT greater than 5)

# Find indices of True values
indices = np.where(x > 5)
print(indices)  # (array([6, 7, 8, 9]),)
print(x[indices])  # [6 7 8 9]

# Modify elements with boolean indexing
x[x > 5] = 0
print(x)  # [0 1 2 3 4 5 0 0 0 0]

# Count True values
print(np.sum(x > 3))  # 2 (number of elements > 3)
print(np.count_nonzero(x > 3))  # 2 (same result)
```

1.7.4 Fancy Indexing (Array Indexing)

```
x = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])

# Index with array of integers
indices = np.array([0, 2, 4])
print(x[indices])  # [10 30 50]

# Can use in any order
indices = np.array([4, 0, 2, 2])
print(x[indices])  # [50 10 30 30] (duplicates allowed)

# 2D fancy indexing
y = np.array([[1,  2,  3,  4],
              [5,  6,  7,  8],
              [9, 10, 11, 12]])

row_idx = np.array([0, 1, 2])
col_idx = np.array([1, 2, 3])
print(y[row_idx, col_idx])  # [2 7 12] (diagonal elements)

# Select specific rows
rows = np.array([0, 2])
print(y[rows])
# [[ 1  2  3  4]
#  [ 9 10 11 12]]
```

Fancy indexing creates copies, not views:

```
a = np.array([1, 2, 3, 4, 5])
b = a[[0, 2, 4]]  # Fancy indexing
b[0] = 99
print(a)  # [1 2 3 4 5] - original unchanged (copy was made)
```

1.8 Array Operations and Broadcasting

1.8.1 Element-wise Operations

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([5, 4, 3, 2, 1])

# Arithmetic operations
print(a + b)   # [ 6  6  6  6  6]
print(a - b)   # [-4 -2  0  2  4]
print(a * b)   # [ 5  8  9  8  5]
print(a / b)   # [0.2  0.5  1.   2.   5. ]
print(a ** 2)  # [ 1  4  9 16 25]
print(a % 3)   # [1 2 0 1 2] (modulo)

# Comparison operations (return boolean arrays)
print(a > 2)   # [False False  True  True  True]
```

```
print(a == b)  # [False False False False False]
print(a <= 3)  # [ True  True  True False False]

# Logical operations on boolean arrays
print((a > 2) & (a < 5))  # [False False  True  True False]
print((a < 2) | (a > 4))  # [ True False False False  True]
```

1.8.2 Universal Functions (ufuncs)

```
a = np.array([0, np.pi/4, np.pi/2, np.pi])

# Trigonometric functions
print(np.sin(a))     # [ 0.   0.71  1.   0. ]
print(np.cos(a))     # [ 1.   0.71  0.  -1. ]
print(np.tan(a))     # [ 0.   1.   Inf  0. ]

# Exponential and logarithmic
b = np.array([1, 2, 3, 4, 5])
print(np.exp(b))     # [  2.72   7.39  20.09  54.60 148.41]
print(np.log(b))     # [0.   0.69 1.10 1.39 1.61]
print(np.log10(b))   # [0.   0.30 0.48 0.60 0.70]
print(np.log2(b))    # [0.   1.   1.58 2.   2.32]

# Power and roots
print(np.sqrt(b))    # [1.   1.41 1.73 2.   2.24]
print(np.power(b, 3))  # [  1   8  27  64 125]

# Rounding
c = np.array([1.2, 2.7, 3.5, -1.8])
print(np.round(c))   # [ 1.  3.  4. -2.]
print(np.floor(c))   # [ 1.  2.  3. -2.]
print(np.ceil(c))    # [ 2.  3.  4. -1.]

# Other useful functions
d = np.array([-1, -2, 3, -4, 5])
print(np.abs(d))     # [1 2 3 4 5]
print(np.sign(d))    # [-1 -1  1 -1  1]
```

1.8.3 Aggregate Functions

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

# Basic statistics
print(a.sum())        # 21 (sum of all elements)
print(a.mean())       # 3.5 (average)
print(a.std())        # 1.71 (standard deviation)
print(a.var())        # 2.92 (variance)
print(a.min())        # 1 (minimum)
print(a.max())        # 6 (maximum)
print(a.argmin())     # 0 (index of minimum)
```

```
print(a.argmax())     # 5 (index of maximum)

# Axis-specific operations
print(a.sum(axis=0))   # [5 7 9] (sum columns)
print(a.sum(axis=1))   # [ 6 15] (sum rows)
print(a.mean(axis=0))  # [2.5 3.5 4.5] (mean of columns)
print(a.mean(axis=1))  # [2. 5.] (mean of rows)

# Cumulative operations
b = np.array([1, 2, 3, 4, 5])
print(np.cumsum(b))    # [ 1  3  6 10 15] (cumulative sum)
print(np.cumprod(b))   # [  1   2   6  24 120] (cumulative product)

# Differences between consecutive elements
print(np.diff(b))      # [1 1 1 1]

# Percentiles
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(np.percentile(data, 25))   # 3.25 (25th percentile)
print(np.percentile(data, 50))   # 5.5 (median)
print(np.percentile(data, 75))   # 7.75 (75th percentile)
print(np.median(data))           # 5.5 (median)
```

1.8.4 Broadcasting

Broadcasting allows NumPy to perform operations on arrays of different shapes. Arrays are automatically expanded to compatible shapes without copying data.

Broadcasting rules:

1. Arrays with different numbers of dimensions: prepend 1s to the shape of the smaller array
2. Dimensions are compatible if they are equal or one of them is 1
3. After broadcasting, each array behaves as if it had shape equal to the element-wise maximum

Examples:

```
# Scalar and array
a = np.array([1, 2, 3])
print(a + 5)  # [6 7 8] - scalar broadcasts to all elements

# 1D array and 2D array
a = np.array([1, 2, 3])      # Shape: (3,)
b = np.array([[10],
              [20],
              [30]])         # Shape: (3, 1)

result = a + b
# [[11 12 13]
#  [21 22 23]
#  [31 32 33]]

# Column and row vectors
```

```
col = np.array([[1], [2], [3]])    # Shape: (3, 1)
row = np.array([[10, 20, 30]])     # Shape: (1, 3)

result = col + row
# [[11 21 31]
#  [12 22 32]
#  [13 23 33]]

# Incompatible shapes (will raise error)
a = np.array([[1, 2, 3]])     # Shape: (1, 3)
b = np.array([[1], [2]])      # Shape: (2, 1)
# result = a + b  # ValueError: shapes not compatible
```

Practical broadcasting example:

```
# Normalize each column of a matrix (zero mean, unit variance)
data = np.random.randn(100, 5)  # 100 samples, 5 features

# Calculate mean and std for each column
mean = data.mean(axis=0)  # Shape: (5,)
std = data.std(axis=0)    # Shape: (5,)

# Normalize (broadcasting applies mean and std to each row)
normalized = (data - mean) / std

print(normalized.mean(axis=0))  # ~[0 0 0 0 0]
print(normalized.std(axis=0))   # ~[1 1 1 1 1]
```

1.9 Array Manipulation Functions

1.9.1 Sorting and Searching

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

# Sort (returns sorted copy)
print(np.sort(a))     # [1 1 2 3 4 5 6 9]

# Sort in place
a.sort()
print(a)              # [1 1 2 3 4 5 6 9]

# Argsort (indices that would sort the array)
b = np.array([3, 1, 4, 1, 5])
indices = np.argsort(b)
print(indices)        # [1 3 0 2 4]
print(b[indices])     # [1 1 3 4 5] - sorted

# Sort 2D array
c = np.array([[3, 1, 4],
              [1, 5, 9]])
print(np.sort(c, axis=0))  # Sort columns
```

```
# [[1 1 4]
#  [3 5 9]]

print(np.sort(c, axis=1))  # Sort rows
# [[1 3 4]
#  [1 5 9]]

# Unique values
d = np.array([1, 1, 2, 2, 2, 3, 4, 4, 5])
print(np.unique(d))   # [1 2 3 4 5]

# Unique with counts
values, counts = np.unique(d, return_counts=True)
print(values)   # [1 2 3 4 5]
print(counts)   # [2 3 1 2 1]
```

1.9.2 Repeating and Tiling

```
a = np.array([1, 2, 3])

# Repeat each element
print(np.repeat(a, 3))        # [1 1 1 2 2 2 3 3 3]
print(np.repeat(a, [1, 2, 3]))  # [1 2 2 3 3 3]

# Tile entire array
print(np.tile(a, 3))          # [1 2 3 1 2 3 1 2 3]

# Tile in 2D
print(np.tile(a, (2, 1)))
# [[1 2 3]
#  [1 2 3]]

print(np.tile(a, (2, 2)))
# [[1 2 3 1 2 3]
#  [1 2 3 1 2 3]]
```

1.9.3 Splitting

```
a = np.arange(12)  # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Split into equal parts
parts = np.split(a, 3)
print(parts)  # [array([0, 1, 2, 3]), array([4, 5, 6, 7]), array([8,
9, 10, 11])]

# Split at specific indices
parts = np.split(a, [3, 7])
print(parts)  # [array([0, 1, 2]), array([3, 4, 5, 6]), array([7, 8,
9, 10, 11])]

# 2D splitting
```

```
b = np.arange(16).reshape(4, 4)
rows = np.vsplit(b, 2)  # Split into 2 parts vertically
cols = np.hsplit(b, 2)  # Split into 2 parts horizontally
```

1.10 Linear Algebra with NumPy

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Element-wise multiplication
print(A * B)
# [[ 5 12]
#  [21 32]]

# Matrix multiplication (Python 3.5+)
print(A @ B)
# [[19 22]
#  [43 50]]

# Alternative matrix multiplication
print(np.dot(A, B))      # Same as @
print(np.matmul(A, B))   # Same as @

# Matrix transpose
print(A.T)
# [[1 3]
#  [2 4]]

# Matrix inverse
print(np.linalg.inv(A))
# [[-2.   1. ]
#  [ 1.5 -0.5]]

# Determinant
print(np.linalg.det(A))  # -2.0

# Matrix rank
print(np.linalg.matrix_rank(A))  # 2

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)

# Solving linear system Ax = b
b = np.array([1, 2])
x = np.linalg.solve(A, b)
print("Solution:", x)
```

```python
# Verify solution
print(np.allclose(A @ x, b))  # True

# Singular Value Decomposition (SVD)
U, s, Vt = np.linalg.svd(A)
print("U:\n", U)
print("s:", s)
print("Vt:\n", Vt)

# Norms
print(np.linalg.norm(A))         # Frobenius norm
print(np.linalg.norm(A, ord=2))  # 2-norm (spectral norm)
```

1.11 Aliasing vs Copying

Critical concept: Understanding when NumPy creates views vs copies prevents unexpected bugs.

1.11.1 Views (Same Data)

Operations that create **views** share data with the original array:

```python
a = np.array([1, 2, 3, 4, 5])

# Assignment creates view
b = a
b[0] = 99
print(a)  # [99  2  3  4  5] - original changed!

# Slicing creates view
c = a[1:4]
c[0] = 77
print(a)  # [99 77  3  4  5] - original changed!

# Reshape creates view
d = a.reshape(5, 1)
d[0, 0] = 88
print(a)  # [88 77  3  4  5] - original changed!

# Transpose creates view
e = np.array([[1, 2], [3, 4]])
f = e.T
f[0, 0] = 99
print(e)  # [[99  2] [ 3  4]] - original changed!

# Ravel creates view (when possible)
g = e.ravel()
g[0] = 77
print(e)  # [[77  2] [ 3  4]] - original changed!
```

1.11.2 Copies (Independent Data)

Operations that create **copies** have independent data:

```
a = np.array([1, 2, 3, 4, 5])

# Explicit copy
b = a.copy()
b[0] = 99
print(a)  # [1 2 3 4 5] - original unchanged

# Flatten creates copy
c = np.array([[1, 2], [3, 4]])
d = c.flatten()
d[0] = 99
print(c)  # [[1 2] [3 4]] - original unchanged

# Fancy indexing creates copy
e = a[[0, 2, 4]]
e[0] = 99
print(a)  # [1 2 3 4 5] - original unchanged

# Boolean indexing creates copy
f = a[a > 2]
f[0] = 99
print(a)  # [1 2 3 4 5] - original unchanged
```

1.11.3 Detecting Views vs Copies

```
a = np.array([1, 2, 3, 4, 5])

# Check if array owns its data
print(a.base is None)  # True - owns its data

# Slice (view)
b = a[1:4]
print(b.base is a)  # True - b is a view of a

# Copy
c = a.copy()
print(c.base is None)  # True - c owns its data
```

Rule of thumb: When in doubt, use `.copy()` to avoid unintended side effects!

Part 2: SciPy - Scientific Python

2.1 What is SciPy?

SciPy is an open-source library built on NumPy that provides high-level scientific algorithms and specialized mathematical functions. It contains optimized implementations from established Fortran and C libraries (BLAS, LAPACK, ODEPACK, FFTPACK). **Standard import:**

```
import scipy
from scipy import stats, signal, optimize, integrate
print(scipy.__version__)  # e.g., '1.11.0'
```

2.2 SciPy Core Modules

SciPy is organized into submodules by functionality:

Module	Functionality
scipy.stats	Statistical functions, distributions, hypothesis tests
scipy.signal	Signal processing (filtering, convolution, spectral analysis)
scipy.optimize	Optimization (minimization, curve fitting, root finding)
scipy.integrate	Numerical integration (ODEs, quadrature)
scipy.interpolate	Interpolation techniques (1D, 2D, splines)
scipy.linalg	Linear algebra (extended beyond NumPy)
scipy.fft	Fast Fourier Transform algorithms
scipy.ndimage	N-dimensional image processing
scipy.spatial	Spatial data structures and algorithms

2.3 SciPy vs NumPy

NumPy provides:

- Basic array operations and data structures
- Core functionality: creation, manipulation, basic math
- Fast element-wise operations and broadcasting
- Foundation required for all scientific Python
- Examples: `np.array()`, `np.mean()`, `np.sum()`

SciPy provides:

- Advanced scientific algorithms built on NumPy
- Specialized tools for specific domains
- Complex algorithms: FFT, integration, filtering
- Task-specific; import only needed modules
- Examples: `stats.ttest()`, `signal.butter()`, `optimize.minimize()`

Key relationship: NumPy provides the infrastructure; SciPy provides the algorithms.

2.4 Statistics Module (scipy.stats)

```python
from scipy import stats
import numpy as np

# Generate sample data
np.random.seed(42)
data1 = np.random.normal(100, 15, 50)
data2 = np.random.normal(105, 15, 50)

# Descriptive statistics
print(stats.describe(data1))
# DescribeResult(nobs=50, minmax=(67.2, 133.7), mean=99.8,
#                variance=224.5, skewness=0.12, kurtosis=-0.43)

# T-test (comparing two groups)
t_stat, p_value = stats.ttest_ind(data1, data2)
print(f"t-statistic: {t_stat:.3f}, p-value: {p_value:.3f}")

# One-sample t-test (comparing to a value)
t_stat, p_value = stats.ttest_1samp(data1, 100)
print(f"t-statistic: {t_stat:.3f}, p-value: {p_value:.3f}")

# Pearson correlation
r, p = stats.pearsonr(data1, data2)
print(f"Correlation: r={r:.3f}, p={p:.3f}")

# Spearman correlation (rank-based, non-parametric)
rho, p = stats.spearmanr(data1, data2)
print(f"Spearman's rho: {rho:.3f}, p={p:.3f}")

# Normality test (Shapiro-Wilk)
stat, p = stats.shapiro(data1)
print(f"Shapiro-Wilk: W={stat:.3f}, p={p:.3f}")

# Chi-square test
observed = np.array([10, 15, 20, 25])
expected = np.array([15, 15, 20, 20])
chi2, p = stats.chisquare(observed, expected)
print(f"Chi-square: χ²={chi2:.3f}, p={p:.3f}")

# Probability distributions
# Normal distribution
x = np.linspace(-3, 3, 100)
pdf = stats.norm.pdf(x, loc=0, scale=1)  # Probability density
cdf = stats.norm.cdf(x, loc=0, scale=1)  # Cumulative distribution

# Generate random samples from distributions
normal_samples = stats.norm.rvs(loc=0, scale=1, size=1000)
uniform_samples = stats.uniform.rvs(loc=0, scale=1, size=1000)
```

2.5 Signal Processing Module (scipy.signal)

```
        from scipy import signal
        import numpy as np

        # Generate sample signal
        fs = 1000  # Sampling frequency (Hz)
        t = np.linspace(0, 1, fs)
        sig = np.sin(2*np.pi*10*t) + 0.5*np.sin(2*np.pi*50*t) +
0.2*np.random.randn(fs)

        # Design filters
        # Butterworth bandpass filter
        lowcut = 5
        highcut = 20
        order = 4
        b, a = signal.butter(order, [lowcut, highcut], btype='band', fs=fs)

        # Apply filter (filtfilt for zero-phase filtering)
        filtered_sig = signal.filtfilt(b, a, sig)

        # Convolution (smoothing)
        window = signal.windows.hann(50)  # Hanning window
        smoothed = signal.convolve(sig, window/window.sum(), mode='same')

        # Find peaks
        peaks, properties = signal.find_peaks(sig, height=0.5, distance=20)
        print(f"Found {len(peaks)} peaks")

        # Correlation
        correlation = signal.correlate(sig, sig, mode='full')

        # Spectral analysis
        # Power spectral density (Welch's method)
        freqs, psd = signal.welch(sig, fs=fs, nperseg=256)

        # Spectrogram
        freqs, times, spectrogram = signal.spectrogram(sig, fs=fs)
```

2.6 Optimization Module (scipy.optimize)

```
        from scipy import optimize
        import numpy as np

        # Curve fitting
        # Define model function
        def exponential(x, a, b, c):
            return a * np.exp(-b * x) + c

        # Generate noisy data
        x_data = np.linspace(0, 4, 50)
```

```
        y_data = exponential(x_data, 2.5, 1.3, 0.5) + 0.2 *
np.random.randn(50)

        # Fit curve
        params, covariance = optimize.curve_fit(exponential, x_data, y_data)
        print(f"Fitted parameters: a={params[0]:.2f}, b={params[1]:.2f}, c=
{params[2]:.2f}")

        # Minimization
        # Find minimum of a function
        def objective(x):
            return (x - 2)**2 + 5

        result = optimize.minimize(objective, x0=0)
        print(f"Minimum at x={result.x[0]:.3f}, f(x)={result.fun:.3f}")

        # Root finding
        # Find where function equals zero
        def func(x):
            return x**3 - 2*x - 5

        root = optimize.root_scalar(func, bracket=[2, 3])
        print(f"Root: x={root.root:.3f}")

        # Least squares (linear regression)
        # Fit y = mx + b
        x = np.array([0, 1, 2, 3, 4])
        y = np.array([0.1, 2.1, 3.9, 6.2, 7.8])

        def residuals(params, x, y):
            m, b = params
            return y - (m*x + b)

        result = optimize.least_squares(residuals, [1, 1], args=(x, y))
        m, b = result.x
        print(f"Linear fit: y = {m:.2f}x + {b:.2f}")
```

2.7 Integration Module (scipy.integrate)

```
        from scipy import integrate
        import numpy as np

        # Numerical integration (quadrature)
        def integrand(x):
            return np.exp(-x**2)

        result, error = integrate.quad(integrand, 0, 1)
        print(f"Integral: {result:.6f} ± {error:.2e}")

        # Multiple integration
        def integrand_2d(y, x):
```

```
        return x * y**2

    result, error = integrate.dblquad(integrand_2d, 0, 1, 0, 1)
    print(f"Double integral: {result:.6f}")

    # Solving ordinary differential equations (ODEs)
    # dy/dt = -2y, y(0) = 1
    def dydt(t, y):
        return -2 * y

    # Solve from t=0 to t=2
    t_span = (0, 2)
    t_eval = np.linspace(0, 2, 100)
    y0 = [1]

    solution = integrate.solve_ivp(dydt, t_span, y0, t_eval=t_eval)
    print(f"Final value: y({solution.t[-1]:.1f}) = {solution.y[0,
-1]:.4f}")
```

2.8 Interpolation Module (scipy.interpolate)

```
    from scipy import interpolate
    import numpy as np

    # 1D interpolation
    x = np.array([0, 1, 2, 3, 4, 5])
    y = np.array([0, 1, 4, 9, 16, 25])

    # Linear interpolation
    f_linear = interpolate.interp1d(x, y, kind='linear')

    # Cubic interpolation
    f_cubic = interpolate.interp1d(x, y, kind='cubic')

    # Evaluate at new points
    x_new = np.linspace(0, 5, 50)
    y_linear = f_linear(x_new)
    y_cubic = f_cubic(x_new)

    # Spline interpolation
    tck = interpolate.splrep(x, y, s=0)  # Smoothing spline
    y_spline = interpolate.splev(x_new, tck)

    # 2D interpolation
    x = np.linspace(0, 4, 5)
    y = np.linspace(0, 4, 5)
    X, Y = np.meshgrid(x, y)
    Z = np.sin(X) * np.cos(Y)

    # (interp2d was removed in SciPy 1.14; use RegularGridInterpolator or
RectBivariateSpline)
```

```
        f_2d = interpolate.RegularGridInterpolator((x, y), Z.T,
method='cubic')

        # Evaluate at new points (a grid of (x, y) pairs)
        x_new = np.linspace(0, 4, 20)
        y_new = np.linspace(0, 4, 20)
        Xn, Yn = np.meshgrid(x_new, y_new)
        Z_new = f_2d((Xn, Yn))      # shape (20, 20)
```

Part 3: Matplotlib - Data Visualization

3.1 What is Matplotlib?

Matplotlib is a comprehensive plotting library for creating static, animated, and interactive visualizations in Python. It provides MATLAB-like plotting interface through the `pyplot` module.

Standard import:

```
        import matplotlib.pyplot as plt
        import numpy as np
```

Display modes:

```
        # Interactive mode (plots update immediately)
        plt.ion()

        # Non-interactive mode (plots shown only with plt.show())
        plt.ioff()

        # Show all figures
        plt.show()

        # Close figures
        plt.close()        # Close current figure
        plt.close('all')   # Close all figures
```

3.2 Basic Plotting

3.2.1 Line Plots

```
        import matplotlib.pyplot as plt
        import numpy as np

        # Generate data
        x = np.linspace(0, 10, 100)
        y1 = np.sin(x)
        y2 = np.cos(x)

        # Simple plot
```

```
        plt.plot(x, y1)
        plt.show()

        # Plot multiple lines
        plt.plot(x, y1, label='sin(x)')
        plt.plot(x, y2, label='cos(x)')
        plt.legend()
        plt.show()

        # Customize appearance
        plt.plot(x, y1, 'r-', linewidth=2, label='sin(x)')   # Red solid line
        plt.plot(x, y2, 'b--', linewidth=2, label='cos(x)')  # Blue dashed
line
        plt.xlabel('x')
        plt.ylabel('y')
        plt.title('Trigonometric Functions')
        plt.legend()
        plt.grid(True)
        plt.show()
```

Line styles and markers:

```
        # Line styles: '-' solid, '--' dashed, '-.' dash-dot, ':' dotted
        # Colors: 'r' red, 'g' green, 'b' blue, 'k' black, 'c' cyan, 'm'
magenta, 'y' yellow
        # Markers: 'o' circle, 's' square, '^' triangle, 'd' diamond, '*'
star, '+' plus

        plt.plot(x, y1, 'ro-', markersize=5)   # Red line with circle markers
        plt.plot(x, y2, 'bs--', markersize=5)  # Blue dashed line with square
markers
        plt.show()

        # Using keyword arguments
        plt.plot(x, y1, color='red', linestyle='-', linewidth=2,
                 marker='o', markersize=3, markevery=10, label='sin(x)')
        plt.legend()
        plt.show()
```

3.2.2 Scatter Plots

```
        # Generate random data
        np.random.seed(42)
        x = np.random.randn(100)
        y = 2*x + np.random.randn(100)

        # Simple scatter
        plt.scatter(x, y)
        plt.xlabel('x')
        plt.ylabel('y')
        plt.show()
```

```
# Customize scatter plot
colors = np.random.rand(100)
sizes = 1000 * np.random.rand(100)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.5, cmap='viridis')
plt.colorbar(label='Color value')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Scatter Plot with Variable Colors and Sizes')
plt.show()
```

3.2.3 Bar Charts

```
# Simple bar chart
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]

plt.bar(categories, values)
plt.xlabel('Category')
plt.ylabel('Value')
plt.title('Bar Chart')
plt.show()

# Grouped bar chart
x = np.arange(len(categories))
values1 = [23, 45, 56, 78, 32]
values2 = [34, 52, 47, 63, 41]

width = 0.35
plt.bar(x - width/2, values1, width, label='Group 1')
plt.bar(x + width/2, values2, width, label='Group 2')
plt.xticks(x, categories)
plt.xlabel('Category')
plt.ylabel('Value')
plt.legend()
plt.show()

# Horizontal bar chart
plt.barh(categories, values)
plt.xlabel('Value')
plt.ylabel('Category')
plt.show()
```

3.2.4 Histograms

```
# Generate random data
data = np.random.normal(100, 15, 1000)

# Simple histogram
plt.hist(data)
```

```
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram')
plt.show()

# Customized histogram
plt.hist(data, bins=30, color='skyblue', edgecolor='black', alpha=0.7)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram with 30 Bins')
plt.show()

# Normalized histogram (probability density)
plt.hist(data, bins=30, density=True, alpha=0.7, label='Data')
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.legend()
plt.show()

# Multiple histograms
data1 = np.random.normal(100, 15, 1000)
data2 = np.random.normal(110, 15, 1000)

plt.hist(data1, bins=30, alpha=0.5, label='Group 1')
plt.hist(data2, bins=30, alpha=0.5, label='Group 2')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

3.3 Figure and Axes

Understanding the matplotlib object hierarchy:

```
x = np.linspace(0, 10, 100)
y1, y2 = np.sin(x), np.cos(x)

# Create figure and axes explicitly
fig, ax = plt.subplots()
ax.plot(x, y1)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Explicit Figure and Axes')
plt.show()

# Multiple subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

ax1.plot(x, y1)
ax1.set_title('Subplot 1')
```

```
ax2.plot(x, y2)
ax2.set_title('Subplot 2')

plt.tight_layout()
plt.show()

# Grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].plot(x, y1)
axes[0, 0].set_title('sin(x)')

axes[0, 1].plot(x, y2)
axes[0, 1].set_title('cos(x)')

axes[1, 0].plot(x, y1**2)
axes[1, 0].set_title('sin²(x)')

axes[1, 1].plot(x, y2**2)
axes[1, 1].set_title('cos²(x)')

plt.tight_layout()
plt.show()
```

3.4 Customizing Plots

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

# Comprehensive customization
fig, ax = plt.subplots(figsize=(10, 6))

# Plot with customization
ax.plot(x, y, 'b-', linewidth=2, label='sin(x)')

# Axes labels and title
ax.set_xlabel('Time (s)', fontsize=12, fontweight='bold')
ax.set_ylabel('Amplitude', fontsize=12, fontweight='bold')
ax.set_title('Customized Plot', fontsize=14, fontweight='bold')

# Axis limits
ax.set_xlim(0, 10)
ax.set_ylim(-1.5, 1.5)

# Grid
ax.grid(True, linestyle='--', alpha=0.7)

# Legend
```

```
        ax.legend(loc='upper right', fontsize=10)

        # Ticks
        ax.tick_params(axis='both', which='major', labelsize=10)

        # Spines (borders)
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

        plt.tight_layout()
        plt.show()

        # Color customization
        colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']
        for i, color in enumerate(colors):
            plt.plot(x, np.sin(x + i*0.5), color=color, label=f'Curve {i+1}')

        plt.legend()
        plt.show()

        # Font customization
        plt.rcParams['font.family'] = 'sans-serif'
        plt.rcParams['font.size'] = 10
        plt.rcParams['axes.labelsize'] = 12
        plt.rcParams['axes.titlesize'] = 14
        plt.rcParams['legend.fontsize'] = 10
```

3.5 Saving Figures

```
        # Save as PNG (raster)
        plt.savefig('figure.png', dpi=300, bbox_inches='tight')

        # Save as PDF (vector)
        plt.savefig('figure.pdf', bbox_inches='tight')

        # Save as SVG (vector)
        plt.savefig('figure.svg', bbox_inches='tight')

        # Save with transparent background
        plt.savefig('figure.png', dpi=300, bbox_inches='tight',
transparent=True)
```

3.6 Advanced Plot Types

3.6.1 Heatmaps

```
        # Create 2D data
        data = np.random.rand(10, 10)

        # Simple heatmap
```

```
plt.imshow(data, cmap='viridis')
plt.colorbar(label='Value')
plt.title('Heatmap')
plt.show()

# Annotated heatmap
fig, ax = plt.subplots()
im = ax.imshow(data, cmap='coolwarm')

# Annotations
for i in range(10):
    for j in range(10):
        text = ax.text(j, i, f'{data[i, j]:.2f}',
                       ha='center', va='center', color='black')

ax.set_title('Annotated Heatmap')
plt.colorbar(im, ax=ax)
plt.show()
```

3.6.2 Contour Plots

```
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

# Filled contours
plt.contourf(X, Y, Z, levels=20, cmap='RdBu_r')
plt.colorbar(label='Z value')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Filled Contour Plot')
plt.show()

# Contour lines
plt.contour(X, Y, Z, levels=10, colors='black', linewidths=0.5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Contour Lines')
plt.show()

# Combined
plt.contourf(X, Y, Z, levels=20, cmap='RdBu_r')
plt.contour(X, Y, Z, levels=10, colors='black', linewidths=0.5)
plt.colorbar(label='Z value')
plt.show()
```

3.6.3 Box Plots

```
# Generate data
data = [np.random.normal(100, 10, 100),
```

```
            np.random.normal(110, 15, 100),
            np.random.normal(105, 12, 100),
            np.random.normal(115, 8, 100)]

    # Box plot
    plt.boxplot(data, tick_labels=['A', 'B', 'C', 'D'])
    plt.ylabel('Value')
    plt.title('Box Plot')
    plt.show()

    # Horizontal box plot
    plt.boxplot(data, tick_labels=['A', 'B', 'C', 'D'], vert=False)
    plt.xlabel('Value')
    plt.title('Horizontal Box Plot')
    plt.show()
```

Additional Resources

NumPy:

- Official documentation: https://numpy.org/doc/stable/
- NumPy quickstart: https://numpy.org/doc/stable/user/quickstart.html
- NumPy for MATLAB users: https://numpy.org/doc/stable/user/numpy-for-matlab-users.html

SciPy:

- Official documentation: https://docs.scipy.org/doc/scipy/
- SciPy tutorial: https://docs.scipy.org/doc/scipy/tutorial/index.html

Matplotlib:

- Official documentation: https://matplotlib.org/
- Pyplot tutorial: https://matplotlib.org/stable/tutorials/introductory/pyplot.html
- Gallery: https://matplotlib.org/stable/gallery/index.html

Books:

- "Python for Data Analysis" by Wes McKinney
- "NumPy Cookbook" by Ivan Idris
- "Matplotlib for Python Developers" by Sandro Tosi

This handout is part of the "Introduction to Scientific Programming" course at CNC-UC, University of Coimbra. For questions or clarifications, please contact the course instructor. **Document**
Version: 1.0
License: CC BY 4.0